

# Normalization in SQL

Normalization is a database design technique used to organize data properly in a database. It helps reduce duplicate data and improves data consistency.

In simple words, normalization means dividing a large table into smaller related tables and connecting them using relationships.

## Why Normalization is Used

Normalization is used to:

- Reduce duplicate data
- Improve data consistency
- Avoid unnecessary data repetition
- Make updating data easier
- Organize database tables properly

Without normalization, databases may contain repeated data, inconsistent values, and maintenance problems.

## Example of Unnormalized Data (UNF)

| StudentID | StudentName | Courses     | Trainer | TrainerPhone |
|-----------|-------------|-------------|---------|--------------|
| 101       | Ravi        | SQL, Java   | Mahesh  | 9876543210   |
| 102       | Kiran       | Python, AWS | Suresh  | 9123456780   |
| 103       | Anu         | SQL, Python | Mahesh  | 9876543210   |

Problems in this table:

- Multiple values are stored in a single cell
- Trainer phone numbers are repeated
- Searching becomes difficult
- Updating data becomes difficult

## First Normal Form (1NF)

A table is in First Normal Form if:

- Each column contains only one value
- No multiple values exist in a single cell

To convert the previous table into 1NF, the Courses column is separated into multiple rows.

## 1NF Table

| StudentID | StudentName | Course | Trainer | TrainerPhone |
|-----------|-------------|--------|---------|--------------|
| 101       | Ravi        | SQL    | Mahesh  | 9876543210   |
| 101       | Ravi        | Java   | Mahesh  | 9876543210   |
| 102       | Kiran       | Python | Suresh  | 9123456780   |
| 102       | Kiran       | AWS    | Suresh  | 9123456780   |
| 103       | Anu         | SQL    | Mahesh  | 9876543210   |
| 103       | Anu         | Python | Mahesh  | 9876543210   |

Now each cell contains only one value.

However, duplicate data still exists because student names and trainer phone numbers are repeated multiple times.

## Second Normal Form (2NF)

A table is in Second Normal Form if:

- It is already in 1NF
- No partial dependency exists

Partial dependency occurs when a column depends only on part of a composite primary key.

In this example, the combination of StudentID and Course acts as a composite key.

StudentName depends only on StudentID.

Trainer depends only on Course.

Because of this, duplicate data is repeated.

To remove partial dependency, the table is divided into separate tables.

## Students Table

| StudentID | StudentName |
|-----------|-------------|
| 101       | Ravi        |
| 102       | Kiran       |
| 103       | Anu         |

Courses Table

| Course | Trainer | TrainerPhone |
|--------|---------|--------------|
| SQL    | Mahesh  | 9876543210   |
| Java   | Mahesh  | 9876543210   |
| Python | Suresh  | 9123456780   |
| AWS    | Suresh  | 9123456780   |

Student\_Courses Table

| StudentID | Course |
|-----------|--------|
| 101       | SQL    |
| 101       | Java   |
| 102       | Python |
| 102       | AWS    |
| 103       | SQL    |
| 103       | Python |

Now student information is stored separately and duplicate student data is reduced.

But trainer phone numbers are still repeated.

### Third Normal Form (3NF)

A table is in Third Normal Form if:

- It is already in 2NF
- No transitive dependency exists

Transitive dependency occurs when one non-key column depends on another non-key column.

In the Courses table:

- Course determines Trainer
- Trainer determines TrainerPhone

So TrainerPhone indirectly depends on Course.

This is called transitive dependency.

To remove it, trainer details are moved into a separate table.

Students Table

| StudentID | StudentName |
|-----------|-------------|
| 101       | Ravi        |
| 102       | Kiran       |
| 103       | Anu         |

Trainers Table

| TrainerID | TrainerName | TrainerPhone |
|-----------|-------------|--------------|
| T1        | Mahesh      | 9876543210   |
| T2        | Suresh      | 9123456780   |

Courses Table

| CourseID | CourseName | TrainerID |
|----------|------------|-----------|
| C1       | SQL        | T1        |
| C2       | Java       | T1        |
| C3       | Python     | T2        |
| C4       | AWS        | T2        |

Student\_Courses Table

| StudentID | CourseID |
|-----------|----------|
| 101       | C1       |
| 101       | C2       |
| 102       | C3       |
| 102       | C4       |
| 103       | C1       |
| 103       | C3       |

Now duplicate trainer phone numbers are removed and the database becomes more organized.

## Difference Between 1NF, 2NF, and 3NF

1NF:

Removes multiple values from a single cell.

2NF:

Removes partial dependency.

3NF:

Removes transitive dependency.

## Conclusion

Normalization helps design clean and efficient databases. It reduces duplicate data, improves consistency, and makes database maintenance easier. Most real-world applications use normalization up to 3NF for better database design.